GESTIÓN DE DATOS EN PYTHON

BBDD

Una base de datos es un software que permite almacenar datos de forma ordenada y estructurada, facilitando su consulta posterior.

Las bases de datos relacionales almacenan datos relacionados entre sí mediante tablas. Separan la estructura lógica (tablas, vistas, índices) del almacenamiento físico, lo que permite gestionar ambos por separado. Son potentes y ampliamente utilizadas (Oracle, MySQL, PostgreSQL, etc.).

El modelo de base de datos orientado a objetos agrupa los datos y métodos en objetos, como en la programación orientada a objetos. Aunque no es tan común como el modelo relacional, también es potente y facilita trabajar con estructuras complejas.

Las bases de datos documentales (NoSQL) almacenan los datos en documentos semiestructurados, sin necesidad de un esquema fijo, lo que les da mayor flexibilidad. Son ideales para escrituras y consultas rápidas (ej: MongoDB, Couchbase).

Clases para el acceso a BD relacionales

Python tiene acceso a gran número de bases datos, la inmensa mayoría están implementadas como un paquete Python que se puede importar y usar. Para un correcto funcionamiento debemos leer cuidadosamente la documentación de cada paquete y ver los ejemplos. Dentro de sos SGBD más populares encontramos:

- SQL Server: pymssql y pyodbc.
- Oracle: cx_Oracle.
- MySQL: Mysql.connector.
- SQLite: sqlite3.
- PostgreSQL: psycopg2.

GESTIÓN DE DATOS EN PYTHON

BBDD

Establecimiento de conexiones con diferentes SGBD

En Python, como en otros lenguajes, existe una propuesta de API estándar para el manejo de bases de datos, de forma que el código sea prácticamente igual independientemente de la base de datos que estemos utilizando por debajo. Esta especificación recibe el nombre de Python Database API o DB-API y se recoge en el PEP 249 (http://www.python.org/dev/peps/pep-0249/).

En el entorno educativo la mayoría de veces se usa MySql como servidor de desarrollo, con lo que vamos a realizar los ejemplos sobre esa base de datos, para tal fin instalaremos el servidor desde la página de descargas (https://www.mysql.com/downloads/) y crearemos una base de datos con la siguiente estructura.

```
CREATE DATABASE IF NOT EXISTS `daw_prog` DEFAULT CHARACTER SET utf8
COLLATE utf8_spanish2_ci;
USE `daw_prog`;
DROP TABLE IF EXISTS `personas`;
CREATE TABLE `personas` (
   `cod` int(11) NOT NULL,
   `nombre` varchar(255) COLLATE utf8_spanish2_ci NOT NULL,
   `apellido_1` varchar(255) COLLATE utf8_spanish2_ci NOT NULL,
   `apellido_2` varchar(255) COLLATE utf8_spanish2_ci NOT NULL,
   `fecha_nacimiento` date NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8 COLLATE=utf8_spanish2_ci;
ALTER TABLE `personas` ADD PRIMARY KEY (`cod`);
COMMIT;
```

GESTIÓN DE DATOS EN PYTHON

BBDD

Establecimiento de conexiones con diferentes SGBD

Instalación del driver Python

```
pip install mysql-connector-python
```

Conexión con la BBDD

```
import mysql.connector
cnn = mysql.connector.connect(user='root', password='', host='127.0.0.1',
                                                        database='daw_prog')
print(cnn)
cnn.close()
```

Gestión de errores de conexión

```
import mysql.connector
from mysql.connector import errorcode
try:
    cnx = mysql.connector.connect(user='root', password='', host='127.0.0.1',
                                                        database='daw_prog')
except mysql.connector.Error as err:
    if err.errno == errorcode.ER_ACCESS_DENIED_ERROR:
        print("Error de conexión")
    elif err.errno == errorcode.ER_BAD_DB_ERROR:
        print("No existe la BBDD")
    else:
        print(err)
else:
    print(cnx)
cnx.close()
```

GESTIÓN DE DATOS EN PYTHON

BBDD

Establecimiento de conexiones con diferentes SGBD

Inserción de datos

```
from datetime import date, datetime, timedelta
import mysql.connector

cnx = mysql.connector.connect(user='root', password='', host='127.0.0.1',
                                                        database='daw_prog')

cursor = cnx.cursor()
add_persona = ("INSERT INTO personas (nombre, apellido_1,
                                  apellido_2, fecha_nacimiento) "
                                  + "VALUES ('Jose', 'Pérez', 'González', '"
                                  + str(date(1977, 6, 14)) + "')")

print(add_persona)
cursor.execute(add_persona)
emp_no = cursor.lastrowid # devuelve el id en un autonumérico
print(emp_no)

cnx.commit()
cursor.close()
cnx.close()
```

GESTIÓN DE DATOS EN PYTHON

BBDD

Establecimiento de conexiones con diferentes SGBD

Borrado de registros

```
import mysql.connector
cnx = mysql.connector.connect(user='root', password='', host='127.0.0.1',
                                                        database='daw_prog')

cnx.autocommit = True
cursor = cnx.cursor()

del_persona = ("DELETE FROM personas WHERE cod = 1")
print(del_persona)

cursor.execute(del_persona)
emp_no = cursor.lastrowid
print(emp_no)

# cnx.commit() Ya no es necesario por el autocommit
cursor.close()
cnx.close()
```

GESTIÓN DE DATOS EN PYTHON

BBDD Establecimiento de conexiones con diferentes SGBD

Ejemplos de consultas sobre la base de datos

```
import mysql.connector
class GenerarEquipo:
    _cnn = None
    _data = None
    def __init__(self):
        try:
            self._cnn = mysql.connector.connect(user='root',
            password='',host='127.0.0.1',database='daw_prog')
            self._cnn.autocommit = True
            self._data = self._cnn.cursor()
            self._data.execute("SELECT * FROM personas")
        except mysql.connector.Error as e:
            print("-", e)
    def __del__(self):
        if self._data is not None:
            self._data.close()
        if self._cnn is not None:
            self._cnn.close()
    def next(self):
        while True:
            row = self._data.fetchone()
            if row is not None:
                yield row
            else:
                return
```

```
misEquipos = GenerarEquipo()
for eq in misEquipos.next():
    print(eq[0], " ", eq[1])
del misEquipos
```